Zomato Analysis

Indian Restaurants: ZOMATO RESTAURANT SUCCESS FACTORS ANALYSIS

In [1]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

In [2]:
```
df=pd.read_csv("Indian-Resturants.csv")
df
```

Out[2]:

	res_id	name	establishment	url	address	city	city_id	locality	latitude	longitude	...	price_range	currency
0	3400299	Bikanervala	['Quick Bites']	https://www.zomato.com/agra/bikanervala-khanda...	Kalyani Point, Near Tulsi Cinema, Bypass Road,...	Agra	34	Khandari	27.211450	78.002381	...	2	Rs.
1	3400005	Mama Chicken Mama Franky House	['Quick Bites']	https://www.zomato.com/agra/mama-chicken-mama-...	Main Market, Sadar Bazaar, Agra Cantt, Agra	Agra	34	Agra Cantt	27.160569	78.011583	...	2	Rs.
2	3401013	Bhagat Halwai	['Quick Bites']	https://www.zomato.com/agra/bhagat-halwai-2-sh...	62/1, Near Easy Day, West Shivaji Nagar, Goalp...	Agra	34	Shahganj	27.182938	77.979684	...	1	Rs.
3	3400290	Bhagat Halwai	['Quick Bites']	https://www.zomato.com/agra/bhagat-halwai-civi...	Near Anjana Cinema, Nehru Nagar, Civil Lines, ...	Agra	34	Civil Lines	27.205668	78.004799	...	1	Rs.
4	3401744	The Salt Cafe Kitchen & Bar	['Casual Dining']	https://www.zomato.com/agra/the-salt-cafe-kitc...	1C,3rd Floor, Fatehabad Road, Tajganj, Agra	Agra	34	Tajganj	27.157709	78.052421	...	3	Rs.
...	...	...	...	...	...	...	...	...	...	...	...	...	...
211939	3202251	Kali Mirch Cafe And Restaurant	['Casual Dining']	https://www.zomato.com/vadodara/kali-mirch-caf...	Manu Smriti Complex, Near Navrachna School, GI...	Vadodara	32	Fatehgunj	22.336931	73.192356	...	2	Rs.
211940	3200996	Raju Omlet	['Quick Bites']	https://www.zomato.com/vadodara/raju-omlet-kar...	Mahalaxmi Apartment, Opposite B	Vadodara	32	Karelibaug	22.322455	73.197203	...	1	Rs.

	res_id	name	establishment	url	address	city	city_id	locality	latitude	longitude	...	price_range	currency
					O B, Karoli Ba...								
211941	18984164	The Grand Thakar	['Casual Dining']	https://www.zomato.com/vadodara/the-grand-thak...	3rd Floor, Shreem Shalini Mall, Opposite Conqu...	Vadodara	32	Alkapuri	22.310563	73.171163	...	2	Rs.
211942	3201138	Subway	['Quick Bites']	https://www.zomato.com/vadodara/subway-1-akota...	G-2, Vedant Platina, Near Cosmos, Akota, Vadodara	Vadodara	32	Akota	22.270027	73.143068	...	2	Rs.
211943	18879846	Freshco's - The Health Cafe	['Café']	https://www.zomato.com/vadodara/freshcos-the-h...	Shop 7, Ground Floor, Opposite Natubhai Circle...	Vadodara	32	Vadiwadi	22.309935	73.158768	...	2	Rs.

211944 rows × 26 columns

Data Overview:

Explore the basic characteristics of the dataset, including dimensions, data types, and missing values.

In [3]:
```
df.head() # Display the first five rows to understand the dataset structure
```

Out[3]:

	res_id	name	establishment	url	address	city	city_id	locality	latitude	longitude	...	price_range	currency	highlights	a
0	3400299	Bikanervala	['Quick Bites']	https://www.zomato.com/agra/bikanervala-khanda...	Kalyani Point, Near Tulsi Cinema, Bypass Road,...	Agra	34	Khandari	27.211450	78.002381	...	2	Rs.	['Lunch', 'Takeaway Available', 'Credit Card',...	
1	3400005	Mama Chicken Mama Franky House	['Quick Bites']	https://www.zomato.com/agra/mama-chicken-mama-...	Main Market, Sadar Bazaar, Agra Cantt, Agra	Agra	34	Agra Cantt	27.160569	78.011583	...	2	Rs.	['Delivery', 'No Alcohol Available', 'Dinner',...	
2	3401013	Bhagat Halwai	['Quick Bites']	https://www.zomato.com/agra/bhagat-halwai-2-sh...	62/1, Near Easy Day, West Shivaji Nagar, Goalp...	Agra	34	Shahganj	27.182938	77.979684	...	1	Rs.	['No Alcohol Available', 'Dinner', 'Takeaway A...	
3	3400290	Bhagat Halwai	['Quick Bites']	https://www.zomato.com/agra/bhagat-halwai-civi...	Near Anjana Cinema, Nehru Nagar, Civil Lines, ...	Agra	34	Civil Lines	27.205668	78.004799	...	1	Rs.	['Takeaway Available', 'Credit Card', 'Lunch',...	
4	3401744	The Salt Cafe Kitchen & Bar	['Casual Dining']	https://www.zomato.com/agra/the-salt-cafe-kitc...	1C,3rd Floor, Fatehabad Road, Tajganj, Agra	Agra	34	Tajganj	27.157709	78.052421	...	3	Rs.	['Lunch', 'Serves Alcohol', 'Cash', 'Credit Ca...	

5 rows × 26 columns

In [4]:
```
df.shape # dimensions
```

Out[4]: (211944, 26)

In [5]:
```
df.dtypes #data types
```

Out[5]:
```
res_id                    int64
name                     object
establishment            object
url                      object
address                  object
city                     object
city_id                   int64
locality                 object
latitude                float64
longitude               float64
zipcode                  object
country_id                int64
locality_verbose         object
cuisines                 object
timings                  object
average_cost_for_two      int64
price_range               int64
currency                 object
highlights               object
aggregate_rating        float64
rating_text              object
votes                     int64
photo_count               int64
opentable_support       float64
delivery                  int64
takeaway                  int64
dtype: object
```

In [6]:
```
df.describe() # Generate summary statistics for numerical features
```

Out[6]:

	res_id	city_id	latitude	longitude	country_id	average_cost_for_two	price_range	aggregate_rating	votes	photo_count	opentable_s
count	2.119440e+05	211944.000000	211944.000000	211944.000000	211944.0	211944.000000	211944.000000	211944.000000	211944.000000	211944.000000	2
mean	1.349411e+07	4746.785434	21.499758	77.615276	1.0	595.812229	1.882535	3.395937	378.001864	256.971224	
std	7.883722e+06	5568.766386	22.781331	7.500104	0.0	606.239363	0.892989	1.283642	925.333370	867.668940	
min	5.000000e+01	1.000000	0.000000	0.000000	1.0	0.000000	1.000000	0.000000	-18.000000	0.000000	
25%	3.301027e+06	11.000000	15.496071	74.877961	1.0	250.000000	1.000000	3.300000	16.000000	3.000000	
50%	1.869573e+07	34.000000	22.514494	77.425971	1.0	400.000000	2.000000	3.800000	100.000000	18.000000	
75%	1.881297e+07	11306.000000	26.841667	80.219323	1.0	700.000000	2.000000	4.100000	362.000000	128.000000	
max	1.915979e+07	11354.000000	10000.000000	91.832769	1.0	30000.000000	4.000000	4.900000	42539.000000	17702.000000	

In [7]:
```
df.isna().sum() #missing values
```

Out[7]:
```
res_id                      0
name                        0
establishment               0
url                         0
address                   134
city                        0
city_id                     0
locality                    0
latitude                    0
longitude                   0
zipcode                163187
country_id                  0
locality_verbose            0
cuisines                 1391
timings                  3874
average_cost_for_two        0
price_range                 0
currency                    0
highlights                  0
aggregate_rating            0
rating_text                 0
votes                       0
photo_count                 0
opentable_support          48
delivery                    0
takeaway                    0
dtype: int64
```

The dataset was explored to understand its structure, dimensions, data types, statistical properties, and missing values. This initial analysis provides a foundation for subsequent data cleaning and exploratory analysis.

In [8]:
```
#filling missing values
df['address'].fillna(df['address'].mode()[0], inplace=True)
```

```
C:\Users\91933\AppData\Local\Temp\ipykernel_8348\2934873277.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained as
signment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a cop
y.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to pe
rform the operation inplace on the original object.


  df['address'].fillna(df['address'].mode()[0], inplace=True)
```

In [9]:
```
df['cuisines'].fillna(df['cuisines'].mode()[0],inplace = True)
```

```
C:\Users\91933\AppData\Local\Temp\ipykernel_8348\2941645637.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained as
signment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a cop
y.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to pe
rform the operation inplace on the original object.


  df['cuisines'].fillna(df['cuisines'].mode()[0],inplace = True)
```

In [10]:
```
df['timings'].fillna(df['timings'].mode()[0], inplace=True)
```

```
C:\Users\91933\AppData\Local\Temp\ipykernel_8348\1908797132.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained as
signment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a cop
y.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to pe
rform the operation inplace on the original object.


  df['timings'].fillna(df['timings'].mode()[0], inplace=True)
```

In [11]:
```
df['opentable_support'].fillna(df['opentable_support'].mode()[0],inplace=True)
```

```
C:\Users\91933\AppData\Local\Temp\ipykernel_8348\158423109.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained ass
ignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a cop
y.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to pe
rform the operation inplace on the original object.


  df['opentable_support'].fillna(df['opentable_support'].mode()[0],inplace=True)
```

In [12]:
```
#removing zipcode as it is not important column with huge null values
df.drop(columns='zipcode', inplace=True)
```

In [13]:
```
df.isna().sum()
```

Out[13]:
```
res_id                  0
name                    0
establishment           0
url                     0
address                 0
city                    0
city_id                 0
locality                0
latitude                0
longitude               0
country_id              0
locality_verbose        0
cuisines                0
timings                 0
average_cost_for_two    0
price_range             0
currency                0
highlights              0
aggregate_rating        0
rating_text             0
votes                   0
photo_count             0
opentable_support       0
delivery                0
takeaway                0
dtype: int64
```

In [14]:
```
#removing dupliacets
df=df.drop_duplicates()
```

In [15]:
```
df.duplicated().sum()
```

Out[15]:
```
np.int64(0)
```

In [16]:
```
df.isna().sum()
```

Out[16]:
```
res_id                  0
name                    0
establishment           0
url                     0
address                 0
city                    0
city_id                 0
locality                0
latitude                0
longitude               0
country_id              0
locality_verbose        0
cuisines                0
timings                 0
average_cost_for_two    0
price_range             0
currency                0
highlights              0
aggregate_rating        0
rating_text             0
votes                   0
photo_count             0
opentable_support       0
delivery                0
takeaway                0
dtype: int64
```

In [17]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 60411 entries, 0 to 211942
Data columns (total 25 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   res_id                60411 non-null  int64
 1   name                  60411 non-null  object
 2   establishment         60411 non-null  object
 3   url                   60411 non-null  object
 4   address               60411 non-null  object
 5   city                  60411 non-null  object
 6   city_id               60411 non-null  int64
 7   locality              60411 non-null  object
 8   latitude              60411 non-null  float64
 9   longitude             60411 non-null  float64
 10  country_id            60411 non-null  int64
 11  locality_verbose      60411 non-null  object
 12  cuisines              60411 non-null  object
 13  timings               60411 non-null  object
 14  average_cost_for_two  60411 non-null  int64
 15  price_range           60411 non-null  int64
 16  currency              60411 non-null  object
 17  highlights            60411 non-null  object
 18  aggregate_rating      60411 non-null  float64
 19  rating_text           60411 non-null  object
 20  votes                 60411 non-null  int64
 21  photo_count           60411 non-null  int64
 22  opentable_support     60411 non-null  float64
 23  delivery              60411 non-null  int64
 24  takeaway              60411 non-null  int64
dtypes: float64(4), int64(9), object(12)
memory usage: 12.0+ MB
```

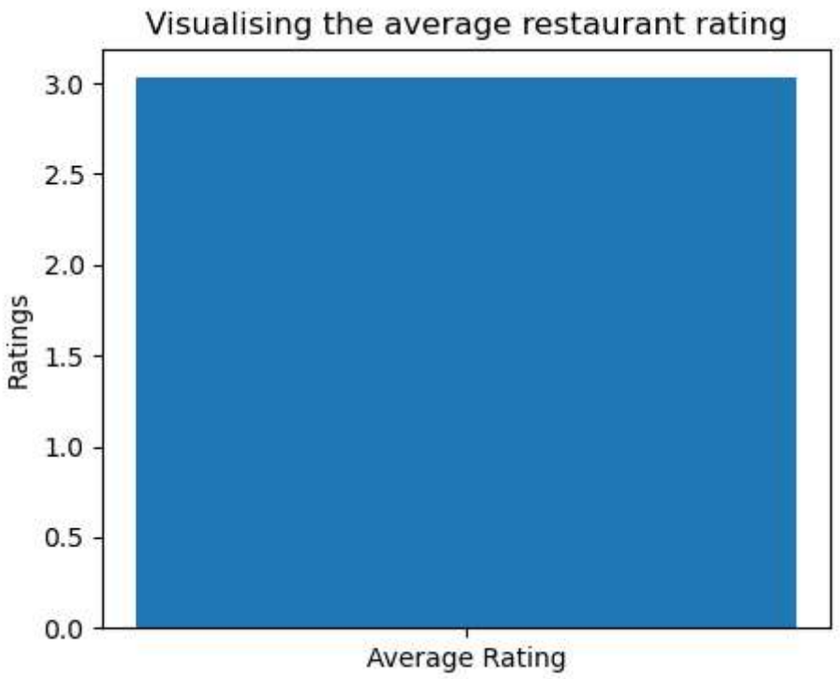
Basic Statistics:

Calculate and visualize the average rating of restaurants.

In [18]:
```
# Calculating average restaurant ratings
avg_rating = df['aggregate_rating'].mean()
print("Average rating of the restaurants is:",avg_rating)
```

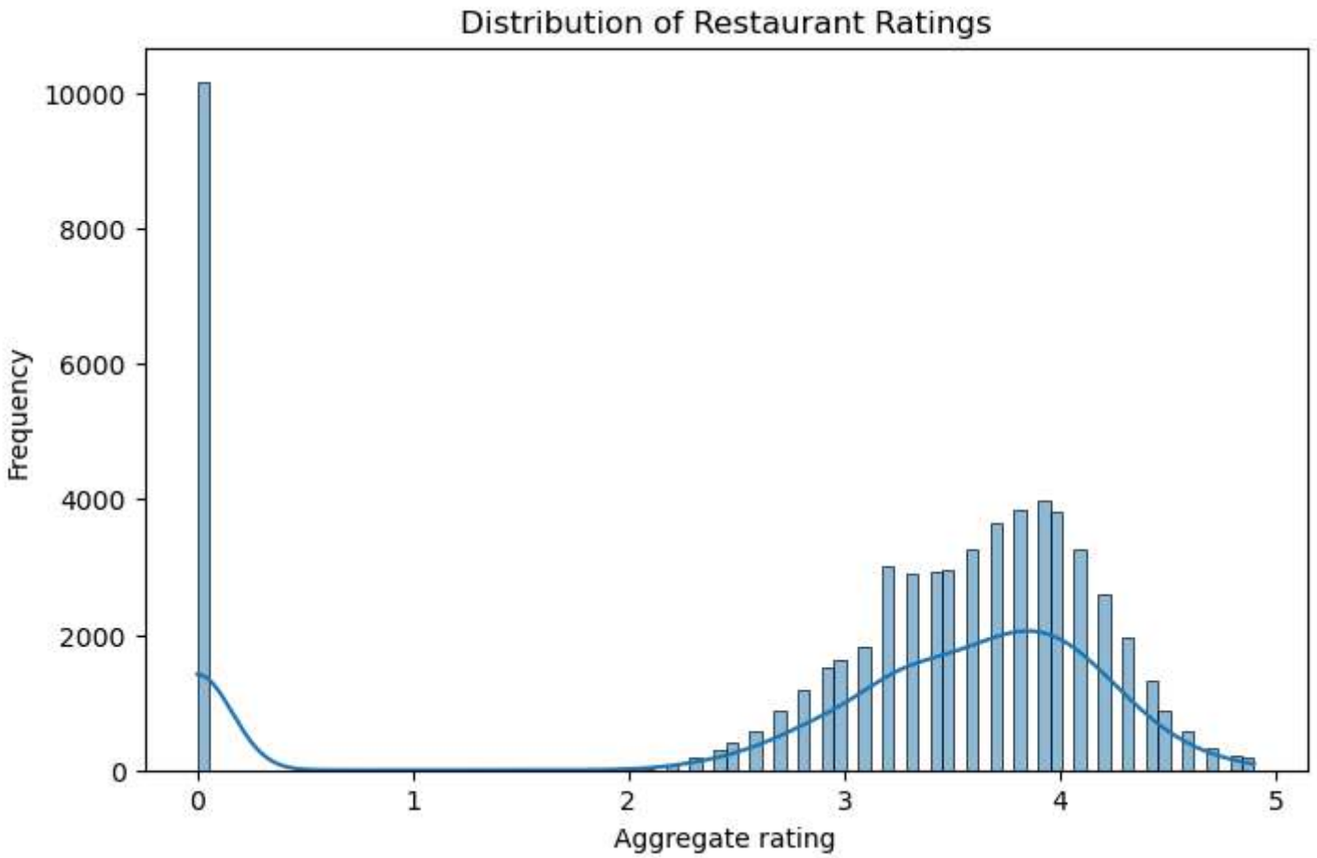
```
Average rating of the restaurants is: 3.032863220274453
```

In [19]:
```
#visualising the average restaurant rating
plt.figure(figsize=(5,4))
# sns.barplot(x=['Average Rating'],y=[avg_rating])
plt.bar('Average Rating',avg_rating)
plt.title("Visualising the average restaurant rating")
plt.ylabel("Ratings")
plt.show()
```



Analyze the distribution of restaurant ratings to understand the overall rating landscape.

In [20]:
```
plt.figure(figsize=(8,5))
sns.histplot(df['aggregate_rating'], kde=True)
plt.title('Distribution of Restaurant Ratings')
plt.xlabel("Aggregate rating")
plt.ylabel("Frequency")
plt.show()
```



The distribution of restaurant ratings shows a significant concentration of unrated restaurants at a rating of 0. Among rated restaurants, most ratings fall between 3.0 and 4.2, with a peak around 3.8–4.0. This indicates that the majority of restaurants receive moderate to high customer ratings, suggesting generally positive customer experiences.

Location Analysis:

Identify the city with the highest concentration of restaurants.

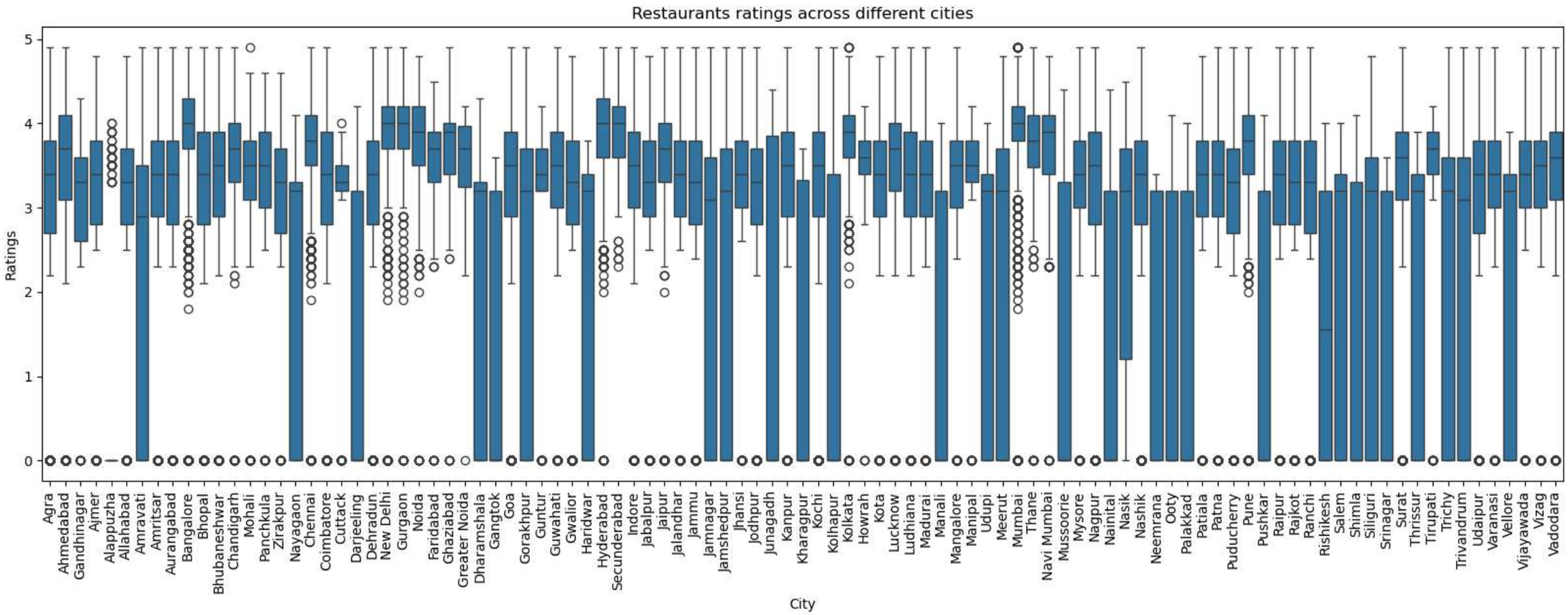
In [21]:
```
# value_counts counts how many time each city appeared
max_restau = df['city'].value_counts().idxmax()
number_restau = df['city'].value_counts().max()
print(f"{max_restau} is the city with highest number of restaurants= {number_restau}")
```

```
Chennai is the city with highest number of restaurants= 2612
```

Visualize the distribution of restaurant ratings across different cities.

In [22]:
```
df['city'].nunique()
plt.figure(figsize=(20,6))
sns.boxplot(x='city', y='aggregate_rating', data=df)
plt.xticks(rotation =90)
plt.title("Restaurants ratings across different cities")
```

```
plt.xlabel("City")
plt.ylabel("Ratings")
plt.show()
```



Observations: Chennai has the highest number of restaurants, ratings vary across different cities

Cuisine Analysis:

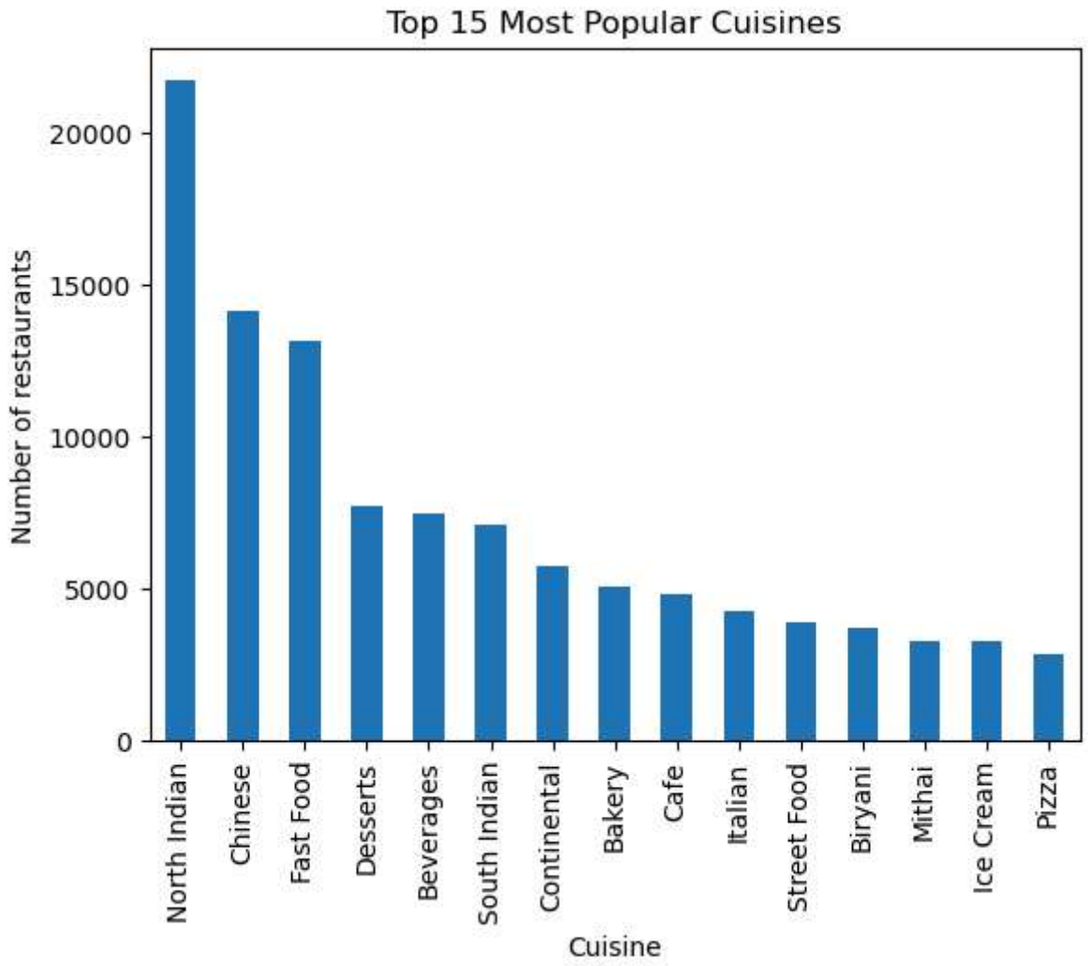
Determine the most popular cuisines among the listed restaurants.

In [23]:
```
separator=df['cuisines'].str.split(", ").explode()
cuisine_analysis=separator.value_counts().head(15) # choosing top 15 because visualisation will not congested
cuisine_analysis
```

Out[23]:
```
cuisines
North Indian    21729
Chinese         14139
Fast Food       13191
Desserts         7755
Beverages        7486
South Indian     7083
Continental      5775
Bakery           5064
Cafe             4804
Italian          4293
Street Food      3933
Biryani          3725
Mithai           3306
Ice Cream        3300
Pizza            2829
Name: count, dtype: int64
```

Observation: Most popular cuisine is North Indian

In [24]:
```
# Visualisation
cuisine_analysis.plot(kind='bar')
plt.title('Top 15 Most Popular Cuisines')
plt.xlabel('Cuisine')
plt.ylabel('Number of restaurants')
plt.show()
```



Observation: North Indian, Chinese, and Fast Food are the most commonly offered cuisines, suggesting they dominate the restaurant market in the dataset.

Investigate if there's a correlation between the variety of cuisines offered and restaurant ratings.

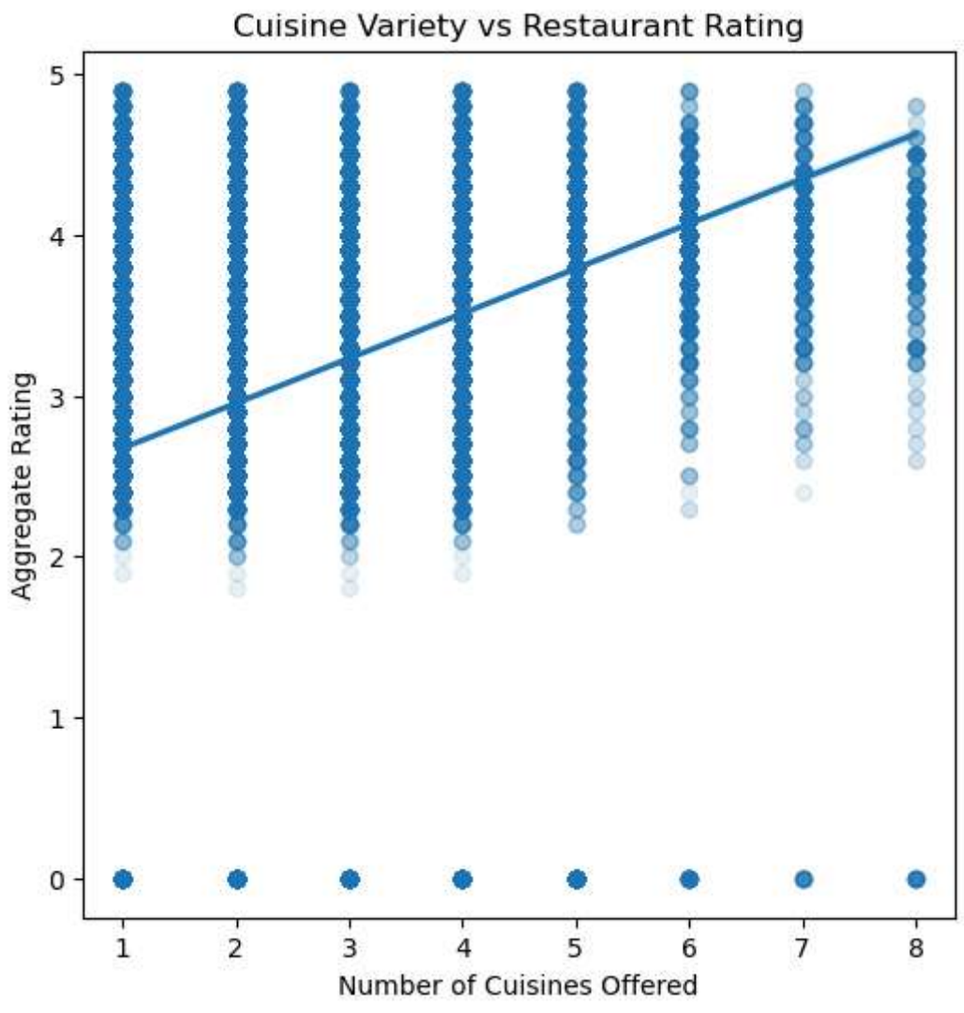
In [25]:
```
#creating cuisine variety column
df['cuisine_var_count'] = df['cuisines'].str.count(", ") + 1
df[['cuisine_var_count', 'aggregate_rating']].corr()
```

```
C:\Users\91933\AppData\Local\Temp\ipykernel_8348\1650255401.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df['cuisine_var_count'] = df['cuisines'].str.count(", ") + 1
```

Out[25]:

	cuisine_var_count	aggregate_rating
cuisine_var_count	1.000000	0.264884
aggregate_rating	0.264884	1.000000

In [26]:
```
#visualizing to make the analysis stronger, using scatter plot for numeric values only
plt.figure(figsize=(6,6))
sns.regplot(x='cuisine_var_count', y='aggregate_rating', data=df, scatter_kws={'alpha':0.1})
plt.title('Cuisine Variety vs Restaurant Rating')
plt.xlabel('Number of Cuisines Offered')
plt.ylabel('Aggregate Rating')
plt.show()
```



Observation: The regression plot indicates a weak positive relationship between the number of cuisines offered and restaurant ratings. Restaurants offering a greater variety of cuisines tend to receive slightly higher ratings, although the relationship is not strong.

Price Range and Rating:

Analyze the relationship between price range and restaurant ratings.

In [27]:
```
# checking the average rating for each price range
price_rest_rating = df.groupby('price_range')['aggregate_rating'].mean()
price_rest_rating
```

Out[27]:
```
price_range
1    2.669885
2    3.281515
3    3.577530
4    3.701439
Name: aggregate_rating, dtype: float64
```

In [28]:
```
plt.figure(figsize=(8,5))
price_rest_rating.plot(kind='bar')
plt.title("Relation between price range and restaurant ratings")
plt.xticks(rotation=0)
plt.xlabel('Price Range')
plt.ylabel('Average Rating')
plt.show()
```

Observation: Restaurants in higher price ranges tend to have higher average ratings. This suggests that customers generally rate premium restaurants more favorably than budget-friendly restaurants.

Visualize the average cost for two people in different price categories.

In [29]:
```
avg_price=df.groupby("price_range")['average_cost_for_two'].mean()
avg_price
```

Out[29]:
```
price_range
1     219.212740
2     524.773848
3    1104.808313
4    2283.108568
Name: average_cost_for_two, dtype: float64
```

In [30]:
```
plt.figure(figsize=(10,4))
avg_price.plot(kind='bar')
plt.title('Average Cost for Two Across Price Categories')
plt.xticks(rotation=0)
plt.xlabel('Price Range')
```

```
plt.ylabel('Average Cost for Two')
plt.show()
```

Observation: The average cost for two people increases substantially with the price range category. Restaurants in the highest price category charge approximately ten times more than those in the lowest price category, indicating that the price range classification accurately reflects restaurant pricing levels.

Online Order and Table Booking:

Investigate the impact of online order availability on restaurant ratings.

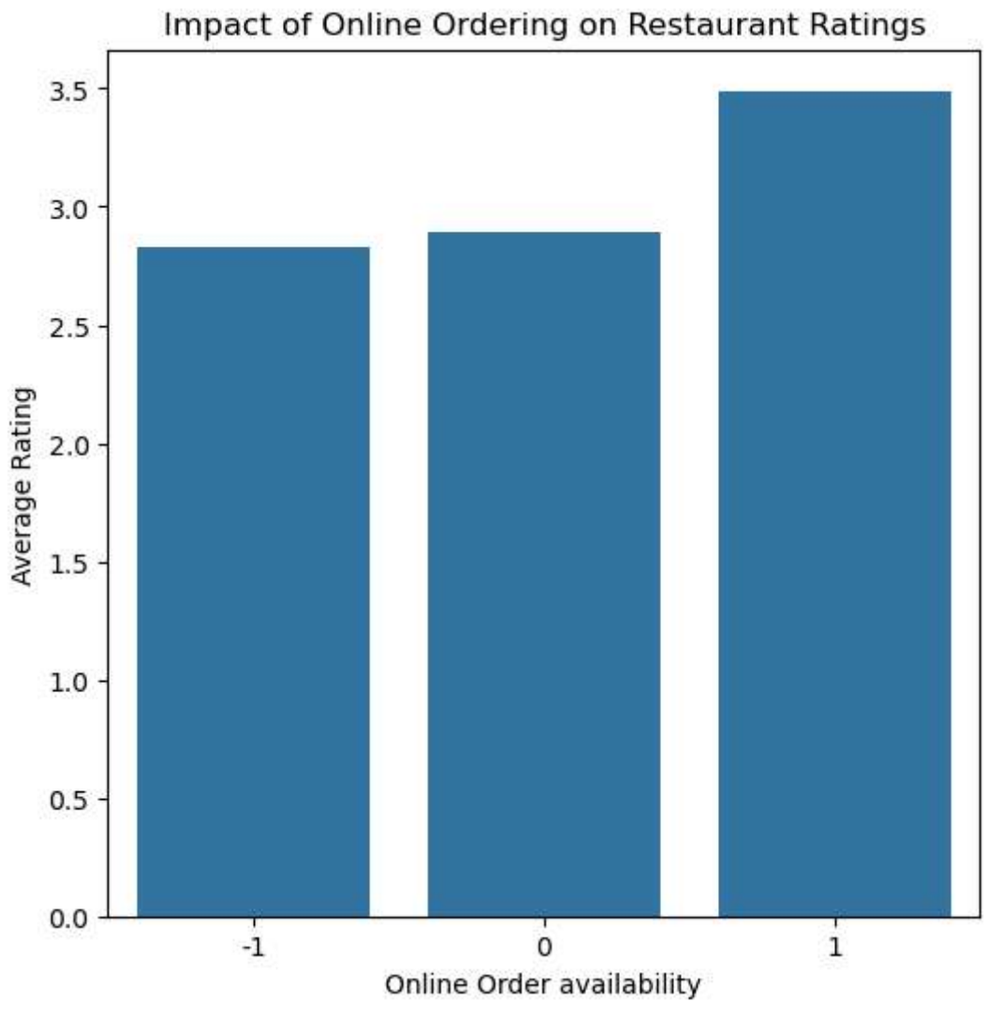
In [31]:
```
df['delivery'].value_counts()
```

Out[31]:
```
delivery
-1    41262
 1    18805
 0      344
Name: count, dtype: int64
```

In [32]:
```
deli_vs_rating = df.groupby('delivery')['aggregate_rating'].mean()
deli_vs_rating
```

Out[32]:
```
delivery
-1    2.825786
 0    2.894767
 1    3.489758
Name: aggregate_rating, dtype: float64
```

In [33]:
```
plt.figure(figsize=(6,6))
sns.barplot(x=deli_vs_rating.index ,y=deli_vs_rating.values )
plt.title("Impact of Online Ordering on Restaurant Ratings")
plt.xlabel("Online Order availability")
plt.ylabel("Average Rating")
plt.show()
```



Obseravtion: Restaurants offering online ordering have significantly higher average ratings (3.49) compared to restaurants without online ordering (2.83), suggesting that online order availability may positively influence customer satisfaction and ratings.

Analyze the distribution of restaurants that offer table booking.

In [34]:
```
df['opentable_support'].value_counts(dropna=False)
```

Out[34]:
```
opentable_support
0.0    60411
Name: count, dtype: int64
```

Observation: The opentable_support column contains only one unique value (0) for all restaurants. Therefore, no meaningful analysis of table booking availability can be performed using this dataset.

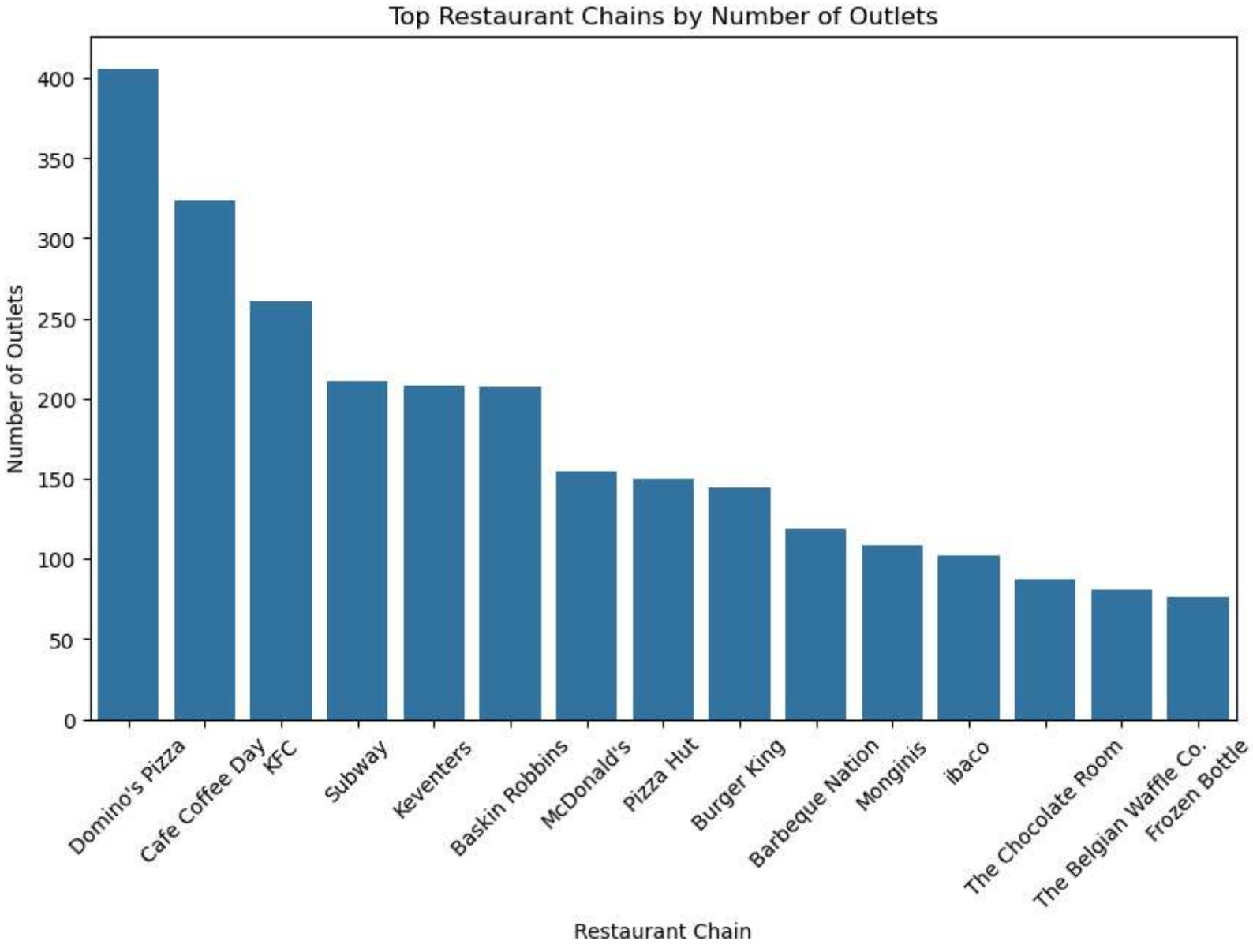
Top Restaurant Chains:

Identify and visualize the top restaurant chains based on the number of outlets.

In [35]:
```
#finding only top 15 because of huge data
top_chain = df['name'].value_counts().head(15)
top_chain
```

Out[35]:
```
name
Domino's Pizza            406
Cafe Coffee Day           323
KFC                       261
Subway                    211
Keventers                 208
Baskin Robbins            207
McDonald's                155
Pizza Hut                 150
Burger King               144
Barbeque Nation           119
Monginis                  108
ibaco                     102
The Chocolate Room         87
The Belgian Waffle Co.     81
Frozen Bottle              76
Name: count, dtype: int64
```

In [36]:
```
plt.figure(figsize=(10,6))
sns.barplot(x=top_chain.index, y=top_chain.values)
plt.title('Top Restaurant Chains by Number of Outlets')
plt.xticks(rotation=45)
plt.xlabel('Restaurant Chain')
plt.ylabel('Number of Outlets')
plt.show()
```



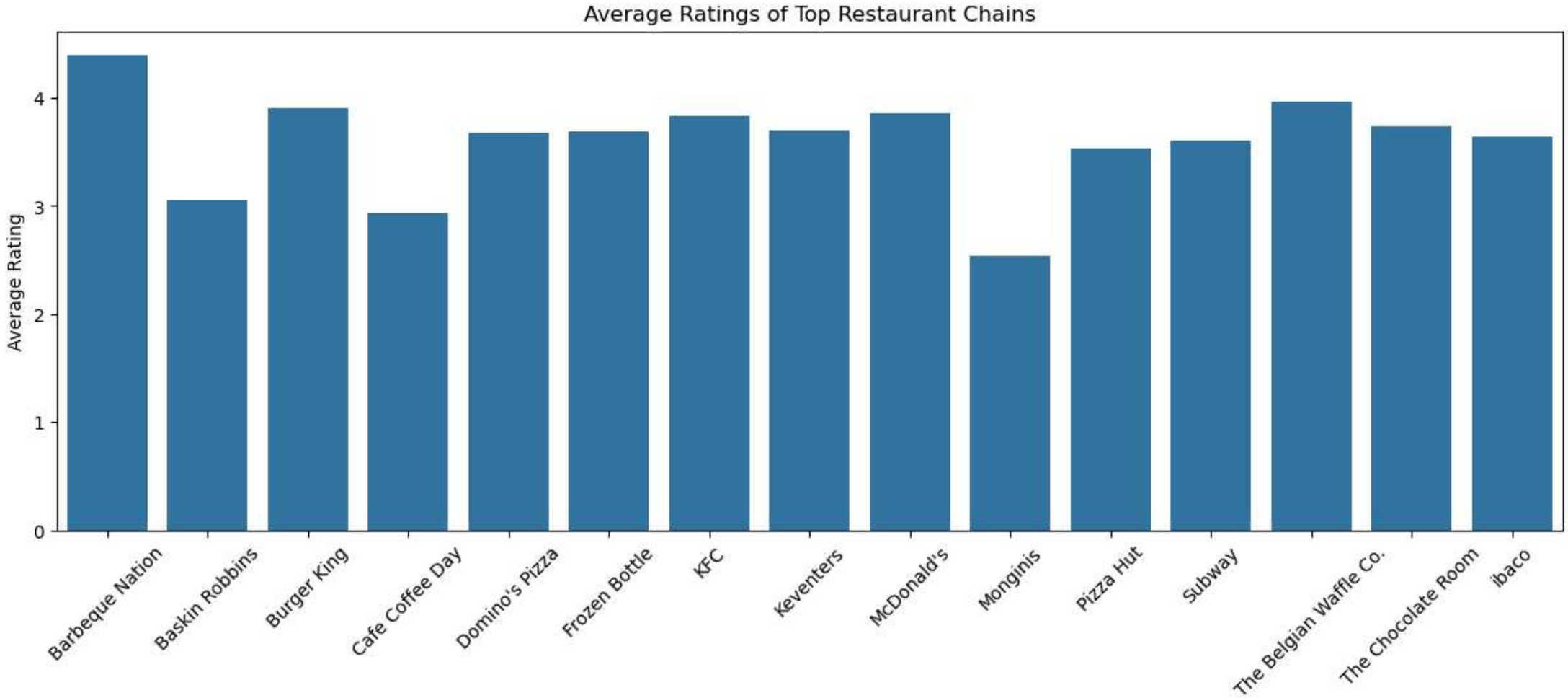
Observation: Domino's Pizza is the largest restaurant chain in the dataset with 406 outlets,

Explore the ratings of these top chains.

In [37]:
```
#we already have top_chain
top_rating = df[df['name'].isin(top_chain.index)]
tr=top_rating.groupby('name')['aggregate_rating'].mean()
tr
```

Out[37]:
```
name
Barbeque Nation          4.395798
Baskin Robbins           3.054106
Burger King              3.902083
Cafe Coffee Day          2.935294
Domino's Pizza           3.669212
Frozen Bottle            3.688158
KFC                      3.831418
Keventers                3.704808
McDonald's               3.849032
Monginis                 2.541667
Pizza Hut                3.535333
Subway                   3.600474
The Belgian Waffle Co.   3.966667
The Chocolate Room       3.737931
ibaco                    3.633333
Name: aggregate_rating, dtype: float64
```

In [38]:
```
plt.figure(figsize=(15,5))
sns.barplot(x=tr.index,
    y=tr.values)
plt.title('Average Ratings of Top Restaurant Chains')
plt.xticks(rotation=45)
plt.xlabel('Restaurant Chain')
plt.ylabel('Average Rating')
plt.show()
```



Observation: Among the top restaurant chains, Barbeque Nation has the highest average rating (4.40), indicating strong customer satisfaction.

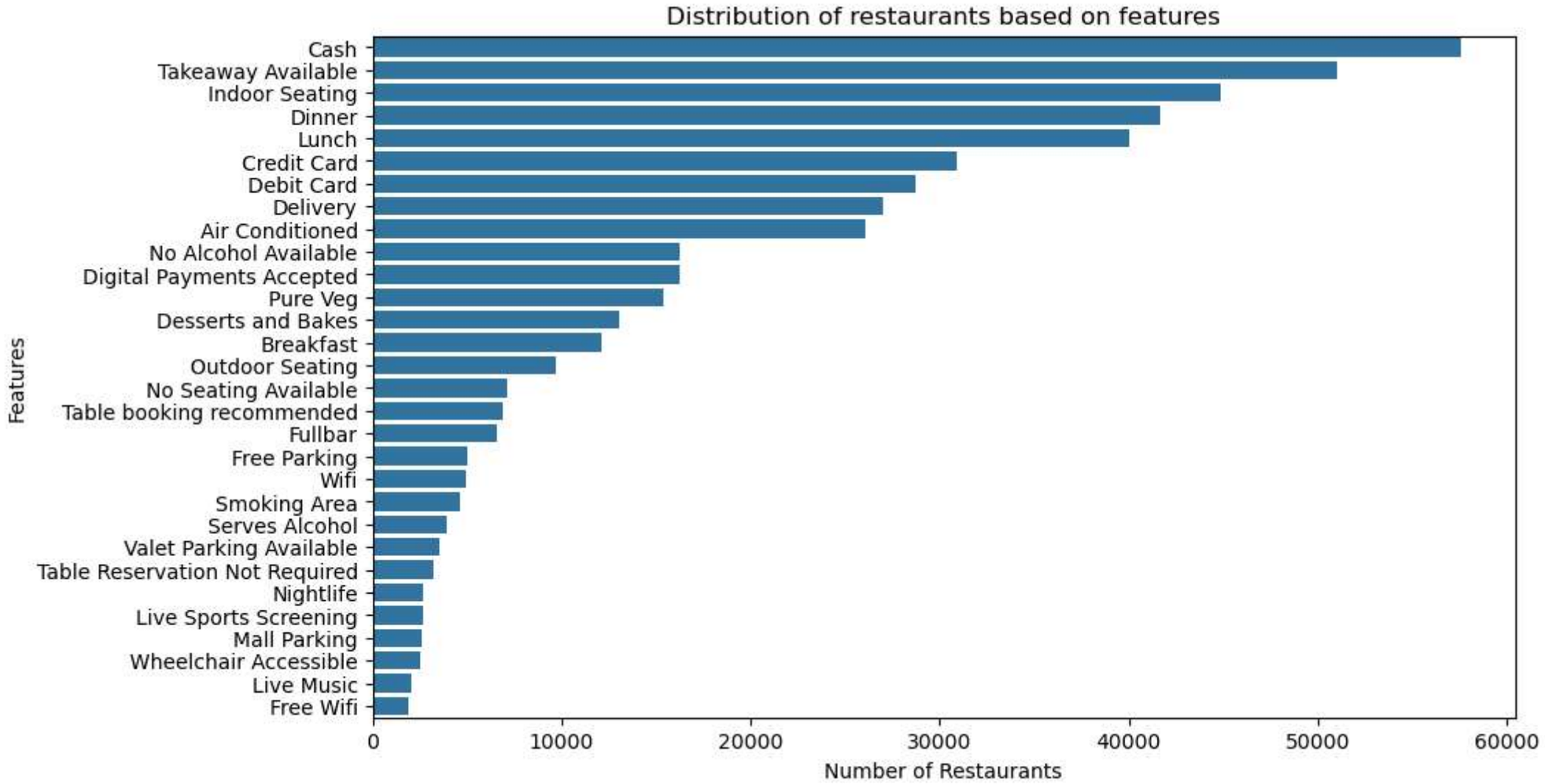
Restaurant Features:

Analyze the distribution of restaurants based on features like Wi-Fi, Alcohol availability, etc.

In [39]:
```
features = df['highlights'].str.strip('[]').str.replace("'", "").str.split(', ').explode()
best_features=features.value_counts().head(30)
best_features
```

Out[39]:
```
highlights
Cash                              57527
Takeaway Available                51005
Indoor Seating                    44842
Dinner                            41680
Lunch                             40008
Credit Card                       30902
Debit Card                        28680
Delivery                          26978
Air Conditioned                   26096
No Alcohol Available              16246
Digital Payments Accepted         16236
Pure Veg                          15356
Desserts and Bakes                13040
Breakfast                         12139
Outdoor Seating                    9708
No Seating Available               7113
Table booking recommended          6922
Fullbar                            6615
Free Parking                       4987
Wifi                               4959
Smoking Area                       4669
Serves Alcohol                     3967
Valet Parking Available            3525
Table Reservation Not Required     3258
Nightlife                          2717
Live Sports Screening              2669
Mall Parking                       2587
Wheelchair Accessible              2495
Live Music                         2041
Free Wifi                          1877
Name: count, dtype: int64
```

In [40]:
```
plt.figure(figsize=(10,6))
sns.barplot(x=best_features.values, y=best_features.index)
plt.title("Distribution of restaurants based on features")
plt.xlabel('Number of Restaurants')
plt.ylabel('Features')
plt.show()
```



Observation: Cash payment, takeaway services, indoor seating, and meal services (lunch/dinner) are the most commonly offered features among restaurants.

Investigate if the presence of certain features correlates with higher ratings.

In [41]:
```
#Only selecting 3 features to investigate: feature 1: serves alcohol
df['Alcohol_serve'] = df['highlights'].str.contains("Serves Alcohol",na=False)
df.groupby('Alcohol_serve')['aggregate_rating'].mean()
```

```
C:\Users\91933\AppData\Local\Temp\ipykernel_8348\2103098062.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df['Alcohol_serve'] = df['highlights'].str.contains("Serves Alcohol",na=False)
```

Out[41]:
```
Alcohol_serve
False    2.984882
True     3.715553
Name: aggregate_rating, dtype: float64
```

In [42]:
```
# feature 1: Allows indoor seating
df.loc[:, 'allowed_indoor_seating']=df['highlights'].str.contains("Indoor Seating",na=False)
df.groupby('allowed_indoor_seating')['aggregate_rating'].mean()
```

```
C:\Users\91933\AppData\Local\Temp\ipykernel_8348\1840924751.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df.loc[:, 'allowed_indoor_seating']=df['highlights'].str.contains("Indoor Seating",na=False)
```

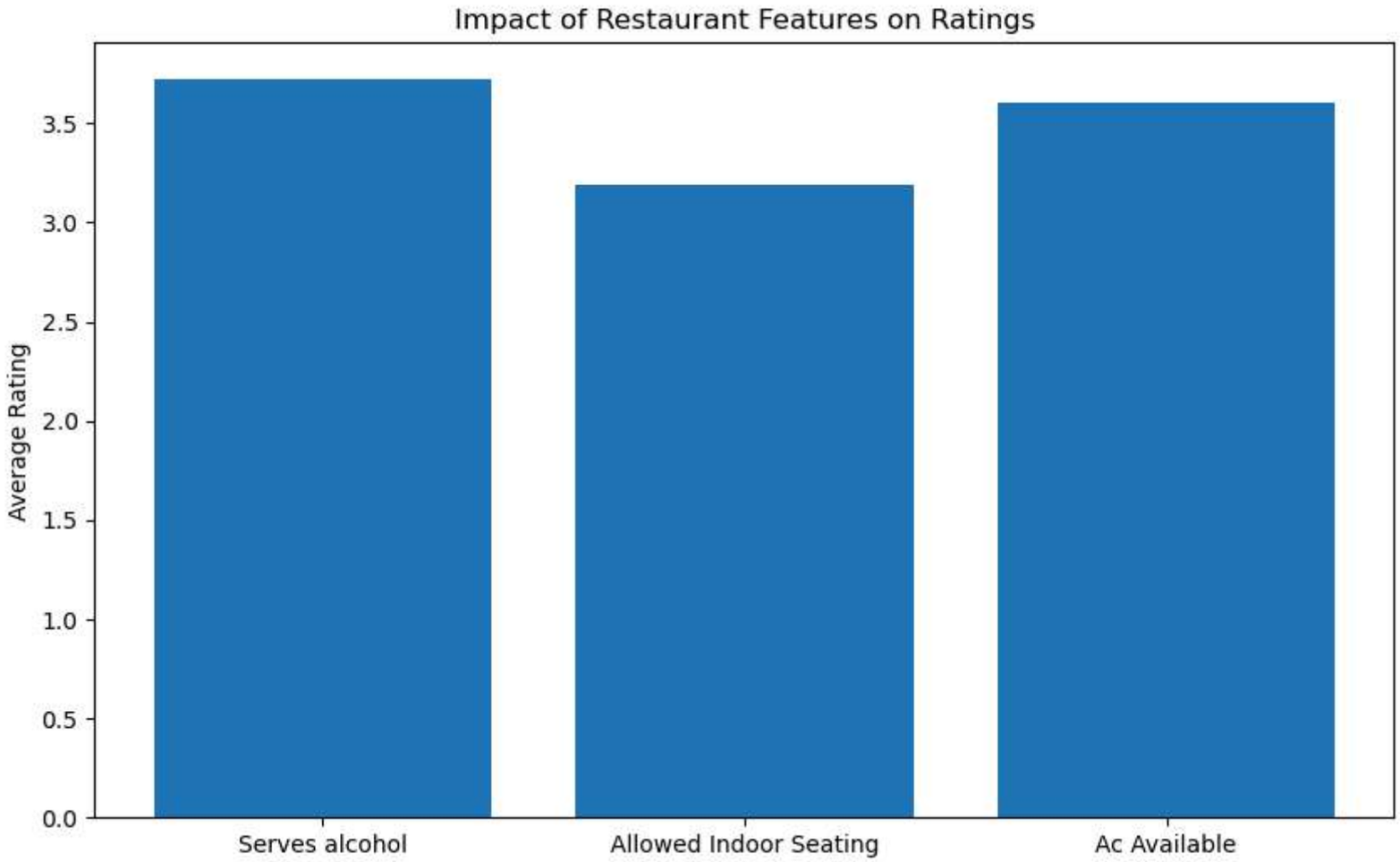
Out[42]:
```
allowed_indoor_seating
False    2.587726
True     3.187414
Name: aggregate_rating, dtype: float64
```

In [43]:
```
#feature3: Ac available
df.loc[:, 'ac_available'] = df['highlights'].str.contains('Air Conditioned',na=False)
df.groupby('ac_available')['aggregate_rating'].mean()
```

```
C:\Users\91933\AppData\Local\Temp\ipykernel_8348\2932208744.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df.loc[:, 'ac_available'] = df['highlights'].str.contains('Air Conditioned',na=False)
```

Out[43]:
```
ac_available
False    2.659685
True     3.523574
Name: aggregate_rating, dtype: float64
```

In [44]:
```
feature_ratings={'Serves alcohol':3.72, 'Allowed Indoor Seating':3.19, 'Ac Available':3.6}
plt.figure(figsize=(10,6))
plt.bar(feature_ratings.keys(), feature_ratings.values())
plt.ylabel('Average Rating')
plt.title('Impact of Restaurant Features on Ratings')
plt.show()
```



Observation: Restaurants offering features like serving alcohol, indoor seating and air conditioning tend to have higher average ratings than rest. Among the selected features, serving alcohol shows higher ratings.

Word Cloud for Reviews:

Create a word cloud based on customer reviews to identify common positive and negative sentiments.

In [45]:
```
# only column that contain reviews equivalent
df['rating_text']
```

Out[45]:
```
0         Very Good
1         Very Good
2         Very Good
3         Very Good
4         Excellent
            ...
211882      Average
211925    Very Good
211926         Good
211940    Very Good
211942         Good
Name: rating_text, Length: 60411, dtype: object
```

In [51]:
```
pip install wordcloud
```

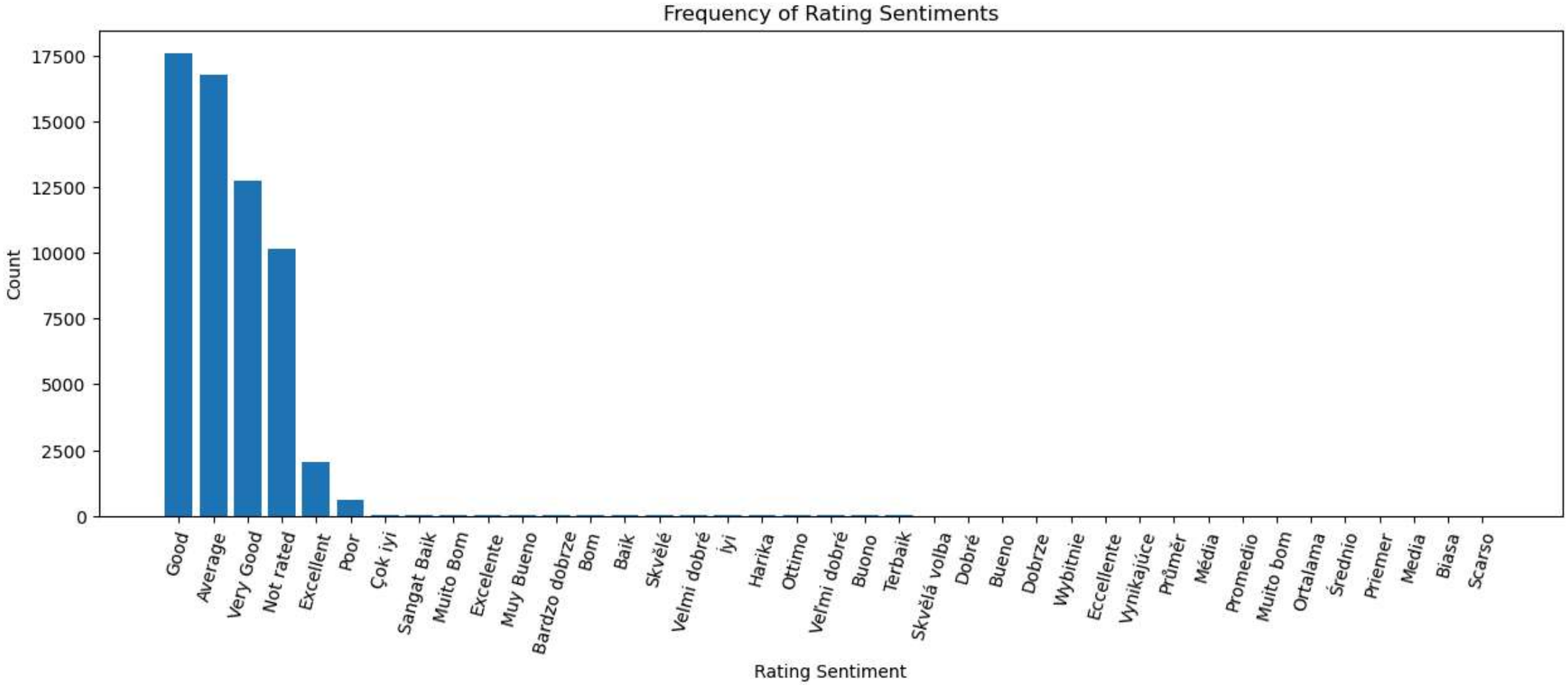
```
Collecting wordcloud
  Downloading wordcloud-1.9.6-cp313-cp313-win_amd64.whl.metadata (3.5 kB)
Requirement already satisfied: numpy>=1.19.3 in c:\users\91933\anaconda3\lib\site-packages (from wordcloud) (2.3.5)
Requirement already satisfied: pillow in c:\users\91933\anaconda3\lib\site-packages (from wordcloud) (12.0.0)
Requirement already satisfied: matplotlib in c:\users\91933\anaconda3\lib\site-packages (from wordcloud) (3.10.6)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\91933\anaconda3\lib\site-packages (from matplotlib->wordcloud) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\91933\anaconda3\lib\site-packages (from matplotlib->wordcloud) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\91933\anaconda3\lib\site-packages (from matplotlib->wordcloud) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\91933\anaconda3\lib\site-packages (from matplotlib->wordcloud) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\91933\anaconda3\lib\site-packages (from matplotlib->wordcloud) (25.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\91933\anaconda3\lib\site-packages (from matplotlib->wordcloud) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\91933\anaconda3\lib\site-packages (from matplotlib->wordcloud) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\91933\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib->wordcloud) (1.17.0)
Downloading wordcloud-1.9.6-cp313-cp313-win_amd64.whl (306 kB)
Installing collected packages: wordcloud
Successfully installed wordcloud-1.9.6
Note: you may need to restart the kernel to use updated packages.
```

In [46]:
```
from wordcloud import WordCloud

text=' '.join(df['rating_text'].astype(str))
wordcloud = WordCloud(width=800, height=400, background_color = 'white').generate(text)
plt.figure(figsize=(10,5))
plt.imshow(wordcloud)
plt.axis('off')
plt.title("Word cloud based on customer review")
plt.show()
```

observation: The word cloud generated from the rating_text column indicates that positive sentiments such as "Excellent" and "Very Good" are more frequent than negative sentiments. This suggests that most restaurants in the dataset received favorable customer ratings, although detailed sentiment analysis was not possible due to the absence of customer review text.

Analyze frequently mentioned words and sentiments.

In [50]:
```
rating_freq = df['rating_text'].value_counts()

plt.figure(figsize=(15,5))
plt.bar(rating_freq.index, rating_freq.values)
plt.xticks(rotation=75)
plt.xlabel('Rating Sentiment')
plt.ylabel('Count')
plt.title('Frequency of Rating Sentiments')
plt.show()
```



obseravtion: Positive sentiments such as "Excellent", "Very Good", and "Good" occur more frequently than negative sentiments,

Seasonal Trends:

Explore if there are any seasonal trends in restaurant ratings or user reviews.

Visualize the distribution of ratings during different times of the year.

Observation: Seasonal Trends Analysis

The dataset does not contain temporal variables such as review dates, months, years, or timestamps that are required to perform seasonal trend analysis. Although the dataset includes a `timings` column, it represents restaurant operating hours rather than chronological or seasonal information. Therefore, seasonal variations in restaurant ratings and customer reviews could not be investigated, and the distribution of ratings across different times of the year could not be visualized.

Conclusion:

Summarized the key findings and insights obtained from the analysis. Provided recommendations for restaurant owners and Zomato users based on the identified success factors. This project structure focused on a specific problem understanding the factors influencing restaurant success and guided us through various aspects of data analysis to derive meaningful insights.

The exploratory data analysis revealed that factors such as cuisine variety, price range, online delivery, restaurant chains, and customer-oriented features significantly influence restaurant ratings and customer preferences. Overall, the analysis provided valuable insights into restaurant characteristics and consumer behavior patterns, while also highlighting certain limitations due to the absence of temporal and review text data.